branches first. It is, of course, not easy to determine what the most promising branch is at any point; a good (although not perfect) indicator could be the distance between target point and split line for a branch: the closest the target, the largest should be the intersection between the nearest neighbor perimeter (the hyper-sphere within which it's still possible to find a closer point to the target) and the region on the other side of the split line. That's the reason why we traverse the tree using a depth-first search: we back-track on the smallest branches first, so, as mentioned, hopefully when we reach larger branches close to the top of the tree, we can prune them.

9.3.6 Region Search

While k-d trees were mainly designed to perform nearest neighbor search, they turned out to be particularly efficient for another kind of operation as well: querying the intersection of our dataset with a given region in the k-dimensional space.

In theory this region could be of any shape, but this operation becomes meaningful only if we can efficiently prune the branches we traverse during the query, and this heavily depends on the region's morphology.

In practice, there are two main cases we are interested in:

- Hyper-spherical regions, as shown in figure 9.25; the geometric interpretation is to query points within a certain distance from a certain point of interest.